

White Box Testing Techniques

Testing the software with the lid off — examining the actual code, paths, and logic inside.

Logic Coverage · Basis Path · Graph Matrices · Loop · Data Flow ·
Mutation

WHY THIS TOPIC EXISTS

Most serious bugs are born in the code itself so at some point, you have to look inside.

Black-box testing checks behaviour from the outside: press a button, check the output. It's essential but it can never tell you that a certain line of code was never run, that a loop breaks on its last pass, or that a variable was used before it was given a value. Those faults live in the internal structure. White-box testing opens the box and tests that structure directly.

The goal: test the program from the inside-out, not just the outside-in.

Glass Box, Clear Box, See-Through Box

White-box testing checks the internal structure of a program — its data structures, code, and logic. Because the tester can see inside, it is also called glass-box, clear-box, or transparent-box testing: the software is treated as a see-through box, not an opaque one.

BLACK BOX

You cannot see inside.

Tests	inputs → outputs only
Based on	requirements / behaviour
Done by	testers
Blind to	which code actually ran

WHITE BOX

You test with the lid off.

Tests	internal code, paths, logic
Based on	the source code structure
Done by	developers
Can prove	every line / branch ran

Why We Need White Box Testing

- | | | |
|---|------------------------------|--|
| 1 | Test both ways | It's better to test a system from the outside-in AND the inside-out — the two views catch different faults. |
| 2 | Bugs hide in the code | Most performance, functional and non-functional issues arise from poor coding — so the code itself must be tested. |
| 3 | Software IS code | An application is built from code; testing that code directly is the only way to be sure each part does its job. |
| 4 | Developers own it | Each developer runs white-box checks to confirm their own code performs its intended functionality. |
| 5 | Essential at scale | If an application must scale to a very large extent, white-box tests become unavoidable, not optional. |

Six White Box Techniques

1

Logic Coverage

Did every statement, branch and condition run?

2

Basis Path Testing

How many independent paths must we test?

3

Graph Matrices

A table that automates path tracing.

4

Loop Testing

Do the loops behave at their boundaries?

5

Data Flow Testing

Is every variable defined before it's used?

6

Mutation Testing

Are our tests good enough to catch faults?

Logic Coverage Criteria

Four questions, each stricter than the last: did every statement, branch, condition, and combination actually run?



Statement Coverage

The most basic goal: make sure every statement (every line) in the source code is executed at least once during testing. To measure it, find the minimum set of paths that visits all the nodes. If some statements are never run, there is a genuine gap in your testing.

$$\text{Coverage} = (\text{statements executed} \div \text{total statements}) \times 100\%$$

It is the bare minimum — necessary, but never sufficient on its own.

EXAMPLE

```
read A, B
IF A > B THEN
    print "A is bigger"
END IF
stop
```

ONE PATH COVERS ALL NODES

Running the program once with A greater than B executes every line here.

So a single test case gives 100% statement coverage — statement coverage = 1.

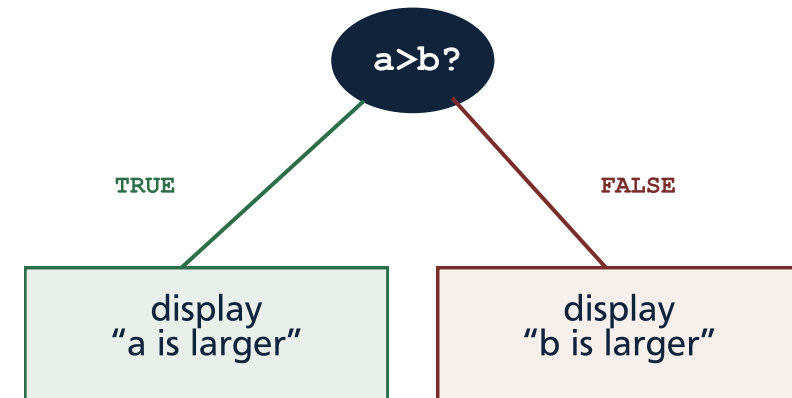
Decision / Branch Coverage

Stricter than statement coverage: every DECISION must be tested both ways — every true and every false outcome of each branch. To measure it, find the minimum number of paths needed to cover all the EDGES of the flow graph. One path can rarely cover them all.

EXAMPLE

```
read a, b
IF a > b THEN
    display "a is larger"
ELSE
    display "b is larger"
END IF
```

TWO PATHS = TWO TEST CASES



Test 1: $a > b$ (true branch).
Test 2: $a \leq b$ (false branch).

Both edges covered — branch coverage = 2.

Condition Coverage

A single decision can contain several conditions joined by AND / OR. Condition coverage requires that each individual condition is tested for BOTH its true and false outcome — not just the decision as a whole.

EXAMPLE

```
i = 1, j = 1;
while ( (i < 5) && (j < 3) )
    // ... body ...
```

Two conditions — (i < 5) and (j < 3) — each tested true and false gives four test cases.

Test	(i < 5)	(j < 3)
1	T	T
2	T	F
3	F	T
4	F	F

All four combinations → condition coverage = 4.

Multiple Condition Coverage

The strictest of the four: test EVERY combination of the truth values of the conditions within a statement — not just each condition once, but all of them together.

EXAMPLE

```
if ( A && B || C && D ) then print C
```

With four conditions there are $2^4 = 16$ possible combinations. Each sub-expression (A&&B and C&&D) must be exercised through every truth value — thorough, but expensive as conditions grow.

A	B	A&&B	C	D	C&&D	Result
T	T	T	F	F	F	T
T	F	F	T	T	T	T
F	T	F	T	F	F	F
F	F	F	F	F	F	F

A sample of the combinations — the full suite tests all 16.

Each Level Is Stricter Than the Last

The four criteria form a ladder — each one demands everything the previous did, and more.

STATEMENT

Every line runs at least once.

Weakest

BRANCH / DECISION

Every decision goes both true and false.

CONDITION

Every individual condition goes true and false.

MULTIPLE CONDITION

Every combination of conditions is tested.

Strongest

Higher coverage finds more faults — but costs more test cases. Teams choose a target level to fit the software's risk.

TECHNIQUE FAMILY 2

Basis Path Testing

Turn the code into a graph, measure its complexity with one formula, and read off the minimum number of tests you need.

2

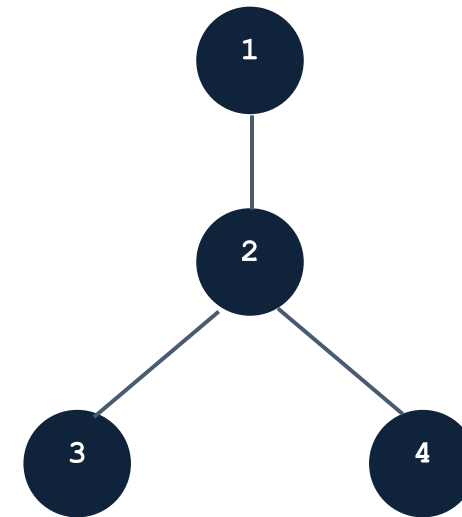
Draw the Control Flow Graph

Basis path testing starts by turning the program (or its flowchart) into a control flow graph — a directed graph where each node is a sequence of statements or a decision point, and each edge is the flow of control from one to the next.

THE THREE STEPS

- 1 Build the control flow graph of the module.
- 2 Compute cyclomatic complexity $V(G) = E - N + 2P$.
- 3 Find that many independent paths — one test case each.

A FLOW GRAPH = NODES + EDGES



Node = statements or a decision.
Edge = the flow between them.
Every IF splits into two edges.

Cyclomatic Complexity

Cyclomatic complexity, $V(G)$, measures how complex a module is — and equals the number of independent paths through it.

$$V(G) = E - N + 2P$$

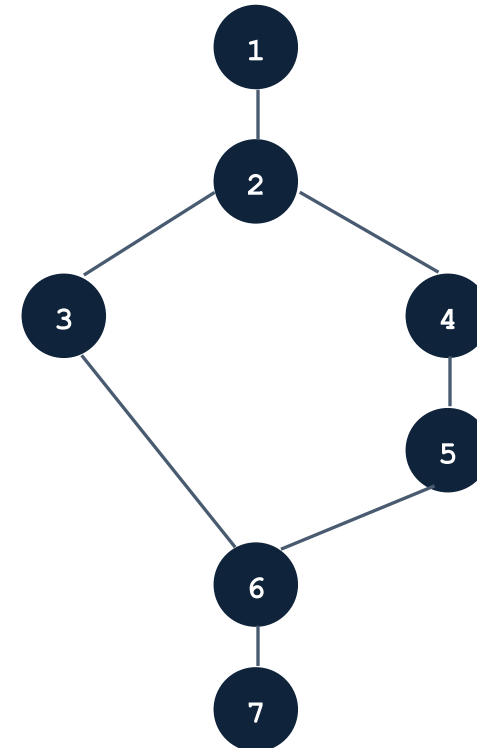
E = number of edges in the flow graph

N = number of nodes

P = number of nodes with exit points

A handy cross-check: $V(G)$ also equals the number of decision points + 1.

WORKED EXAMPLE



The Answer: How Many Tests You Need

For the example graph: $E = 8$ edges, $N = 7$ nodes, $P = 1$.

$$V(G) = 8 - 7 + 2(1) = 3 \rightarrow 3 \text{ independent paths} \rightarrow 3 \text{ test cases}$$

It's a number you can trust

Cyclomatic complexity gives an objective count of the minimum test cases needed for full path coverage — no guessing.

It flags risky code early

A high $V(G)$ means many paths and high complexity: harder to test, more likely to hide bugs, a candidate for refactoring.

It scales to real modules

The same three steps — graph, formula, paths — work whether a function has 3 paths or 30.

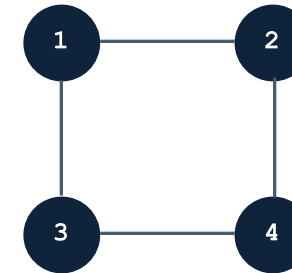
Graph Matrices

Tracing paths by hand on a flow graph is slow and error-prone. A graph matrix is a data structure that lets a tool do it automatically. It is a SQUARE matrix whose size equals the number of nodes; each row and column is a node, and each entry marks an edge (a connection) between two nodes.

CONNECTION MATRIX

A refinement: instead of just marking “edge exists,” each edge is stored with a weight. Those weights (for example, the condition on a branch) are held in the matrix in place of the plain edge — richer information for generating basis-path test cases.

A 4-NODE GRAPH AND ITS MATRIX



	1	2	3	4
1	.	1	1	.
2	.	.	.	1
3	.	.	.	1
4	.	.	.	.

A “1” at row i , column j means an edge runs from node i to node j .

Loop Testing

Loops are everywhere — and they break at the edges. Test the first pass, the last pass, and the pass that never happens.



Test a Loop at Its Boundaries

Loops appear in almost every algorithm, so they must be tested — and errors typically occur at the very beginning or the very end of the loop condition. For a simple loop, run these five checks:

- 1 Is the loop control variable initialised correctly?
- 2 Run the statements inside the loop at least once.
- 3 Run the loop exactly twice.
- 4 Run the loop for the maximum number of iterations.
- 5 Use a value that skips the loop entirely (zero passes).

WHY THE EDGES?

A loop that should run 10 times but stops at 9 — or runs 11 — is the classic off-by-one bug. It only shows on the FIRST pass, the LAST pass, or the skipped pass. The middle iterations almost never reveal it, which is exactly why the five checks target the edges.

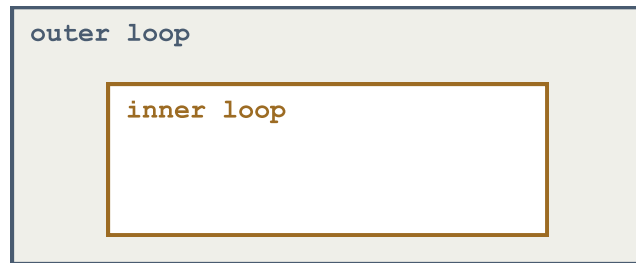
SKIP-THE-LOOP CASE

```
for i = 1 to N
    total = total + i
// what if N = 0 ?
```

When Loops Meet Other Loops

NESTED LOOPS

A loop inside another loop.

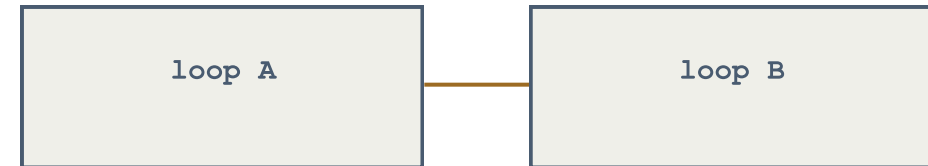


Approach:

Start with the innermost loop, holding all outer loops at their minimum. Test it fully, then work outward one loop at a time until the outermost is reached.

CONCATENATED LOOPS

Separate loops, one after another.



Approach:

The loops are disjoint (not nested), so test each one separately, applying all the simple-loop checks to each in turn.

Data Flow Testing

Follow each variable's life story: was it given a value before it was used? Misused data is a bug waiting to happen.



Track Every Variable's Life

Data flow testing looks for the improper use of data values — faults programmers introduce unintentionally, such as using a variable that was never declared, or one declared but never given a value. An uninitialised variable holds a garbage value, producing unexpected output.

THE CLASSIC BUG

```
int x;  
while ( x < 5 )  
    // ... statements ...  
// x was never initialised!
```

x has no value — the loop's behaviour is unpredictable.

A DATA OBJECT MOVES THROUGH THREE STATES



Defined

The variable is declared and given an initial value.



Used

The variable's value is read — in a computation (c-use) or in a condition (p-use).



Killed

The variable's value is released, or its scope ends.

Definitions, Uses, and the Paths Between

DEFINITION NODE

A statement that gives a variable a value — an input, an assignment, a loop-control statement, or a function call.

USAGE NODE

A statement that reads a variable — an output, a conditional, or a loop-control statement.

du-PATH

A definition-use path: a path from where a variable is defined to where it is used (c-use or p-use).

dc-PATH

A definition-clear path: a du-path with NO other definition of that variable in between — the value survives untouched.

The test goal: for every variable, exercise a clean path from each definition to each use — proving the value that arrives is the value that was set.

TECHNIQUE FAMILY 5

Mutation Testing

A test for your tests: deliberately break the code, and check whether your test suite is sharp enough to notice.

5

Break the Code on Purpose

Mutation testing measures how GOOD your test data is. The code is deliberately mutated — small faults are introduced — creating many versions called mutants, each containing one error. The test suite is then run against them.

Test FAILS on the mutant → good! The fault was caught — the mutant is “killed.”

Test PASSES on the mutant → warning. Your tests missed a real fault — the test data is too weak.

THE PROCESS

- 1** Introduce one small fault into the code → a mutant.
- 2** Run the existing test data against the mutant.
- 3** If a test fails, the mutant is killed — tests are effective.
- 4** If no test fails, strengthen the test data and repeat.

Primary & Secondary Mutants

PRIMARY MUTANT

A single change to the code.

ORIGINAL

```
if (a > b)
    x = x + y;
```

MUTANTS (one change each)

```
M1:  x = x - y;
M2:  x = x + 1;
M3:  x = x + z;
```

SECONDARY MUTANT

More than one change at a time.

ORIGINAL

```
if (a < b)
    a = a + 1;
```

MUTANTS (multiple changes)

```
M1:  if (a < b + 1)
        a = a + 2;
M2:  if (b < a)
        a = a + b;
```